iPhone 6 Plus Android Storage Bring-up Notes

Device tested: iPhone 6 Plus, iPhone7,1, N56AP, t7000, iOS 12.5.8.

What the phone proved

The iOS SSH ramdisk can see and read the internal NAND. It exposes:

- ASPStorage matched on ANSEndpoint1
- ASPBlockStorage
- IONANDBlockDevice
- Apple NAND Media as disk0
- 128 GB Toshiba NAND
- 4096-byte block size
- GPT/APFS container at disk0s1

The first block of /dev/rdisk0 reads successfully and contains a GPT protective MBR ending in 55 aa.

Why Linux does not see the iPhone storage yet

The Hoolock Linux boot only shows RAM disks (/dev/ram0 through /dev/ram15). That means Linux is not reaching the internal NAND as a block device at all.

This is not an APFS format problem yet. APFS only matters after Linux has a real block device like /dev/sda, /dev/mmcblk0, or an Apple NAND equivalent. Right now it has no internal storage block device.

The iPhone 6 Plus storage path is:

```
ans@8040000
  iop-ans-nub
    RTBuddy
      ANSEndpoint1
        ASPStorage
          ASPBlockStorage
            IONANDBlockDevice
              Apple NAND Media -> disk0
```

Important device-tree facts from the live phone:

- ans@8040000
- compatible: iop,s5l8960x
- role: ANS
- interrupts: 0x25, 0x24, 0x27, 0x26
- register region: address 0x208040000, length 0x2000
- nub compatible: iop-nub,rtbuddy

What does not solve it

- A generic Android GSI by itself does not make the storage appear.
- The supplied recovery.img is a Qualcomm Android boot image, so its kernel is not bootable on Apple A8. Its ramdisk/userspace can be useful, but the kernel must still be the iPhone Linux kernel.
- Enabling APFS support alone is not enough.
- The existing CONFIG_NVME_APPLE=y path is for newer Apple ANS/NVMe controllers, not

this A8 ASP/IONAND path as currently wired.

Next real target

The next useful engineering target is an Apple A7/A8 IOP/RTBuddy/ASP storage driver or bridge for Linux:

1. Describe ans@8040000 in the T7000 device tree.
2. Bring up the A7 IOP RTBuddy endpoint for ANSEndpoint1.
3. Implement enough ASPStorage/ASPBlockStorage commands to expose a read-only Linux block device.
4. Only after a block device exists, deal with APFS or place Android on a separate ext4-backed area.

Until step 3 exists, Android cannot boot from the internal iPhone storage.

2026-05-18 phone-side ANS probe test

Built and booted a custom Hoolock Linux payload with:

- drivers/soc/apple/t7000-ans-probe.c
- a T7000 DT node at /soc/ans@208040000
- compatible strings apple,t7000-ans-iop-probe and apple,s5l8960x-ans-iop

The iPhone 6 Plus booted the payload through Pongo and exposed the Hoolock Linux USB shell.

Confirmed from the phone:

- /sys/firmware/fdt contains ans@208040000
- /proc/device-tree/soc/ans@208040000 exists
- /sys/bus/platform/devices/208040000.ans exists
- /sys/bus/platform/drivers/apple-t7000-ans-probe exists
- 208040000.ans is bound to apple-t7000-ans-probe

Storage still does not appear:

```
/proc/partitions:
  ram0..ram15
  mtdblock0  m1n1_stage2.log
  mtdblock1  adt
```

Conclusion: the DTB/boot path is now proven. Linux can receive and enumerate an ANS node, but this probe does not implement the RTBuddy/ASP block protocol. The next blocker is a real A8 ANS/ASP storage driver, not Android userspace or APFS.

2026-05-18 ANS MMIO diagnostic test

Built and booted a second diagnostic payload that maps /soc/ans@208040000, keeps the runtime PM domain active, and exposes sysfs attributes on the phone.

Phone-side result:

```
/sys/bus/platform/devices/208040000.ans/uevent:
  DRIVER=apple-t7000-ans-probe
  OF_FULLNAME=/soc/ans@208040000
  OF_COMPATIBLE_0=apple,t7000-ans-iop-probe
  OF_COMPATIBLE_1=apple,s5l8960x-ans-iop

/sys/bus/platform/devices/208040000.ans/state:
```

```
  mmio_size=0x2000 pm_ret=0 runtime_status=active
/sys/bus/platform/devices/208040000.ans/regs:
  000: 00000001
  004: 00000000
  008: 00000000
  00c: 00000000
  010: 00000000
  014: 00000000
  018: 00000000
  01c: 00000000
  020: 00000001
  024: 00000403
  028: 00000000
  02c: 00000000
  030: 00000000
  034: 00000030
  038: 00000000
  03c: 00000000
```

Conclusion: Linux can power the A8 ANS block and safely read its MMIO window. The next step is not another DTB or Android image test. It is implementing the IOP mailbox/RTBuddy protocol for this controller, then using the ASPStorage endpoint to register a Linux block device.

2026-05-18 ANS mailbox-layout diagnostic

Built and booted a third diagnostic payload that exposes likely ASC and M3 mailbox register offsets from the ANS MMIO window.

Phone-side result:

```
/sys/bus/platform/devices/208040000.ans/state:
  mmio_size=0x2000 pm_ret=0 runtime_status=active
/sys/bus/platform/devices/208040000.ans/mbox:
  asc_a2i_control  @110: 00000000
  asc_i2a_control  @114: 00000000
  asc_a2i_send0    @800: 0018001800000060
  asc_a2i_send1    @808: 6000000300000000
  asc_i2a_recv0    @830: 0000000000000000
  asc_i2a_recv1    @838: 0000000000000000
  m3_irq_enable    @048: 00000000
  m3_irq_ack       @04c: 00000000
  m3_a2i_control   @050: 00000000
  m3_a2i_send0     @060: 0000000000000000
  m3_a2i_send1     @068: 0000000000000000
  m3_i2a_control   @080: 00000000
  m3_i2a_recv0     @0a0: 0000000000000000
  m3_i2a_recv1     @0a8: 0000000000000000
```

Interpretation: the ASC-like mailbox window is present and has non-zero A2I send registers, but its control register values do not line up cleanly with the existing apple-mailbox driver’s empty/full bit expectations. Do not blindly bind the existing RTKit stack yet. Next safe step is an A8-specific mailbox shim that logs control bits and can perform one controlled receive poll before any attempt to send RTBuddy management messages.

2026-05-18 A8 AKF mailbox test

The newer ASC/RTKit wake path did not produce a HELLO on A8:

```
cpu_before=00000000 cpu_after=00000000 a2i_ctl_before=00000000
send msg0=0060000000000220 msg1=0000000000000000
recv0/recv1 remained zero for 32 samples
```

Local hoolock-m1n1/src/akf.c shows that iop,s5l8960x devices use the older AKF mailbox at base + 0x1000, with 32-bit message halves. A fourth diagnostic payload exposed that AKF window and tested it on the phone.

Phone-side AKF state:

```
akf_set          @1000: 00001111
akf_clr          @1004: 00001111
akf_a2i_control  @1008: 00023301
akf_a2i_send0    @1010: 00000001
akf_a2i_send1    @1014: 00600000
akf_a2i_recv0    @1018: 000ff013
akf_a2i_recv1    @101c: 06000000
akf_i2a_control  @1020: 00020001
akf_i2a_recv0    @1038: 00000012
akf_i2a_recv1    @103c: 06000000
```

Polling the AKF receive side produced real ANS/RTBuddy management traffic:

```
0600000000000012
0600000000000004
00b0000000000010
0070000000000001
```

Sending the old-format wake message through akf_send_raw:

```
echo 00000220 06000000 > /sys/bus/platform/devices/208040000.ans/akf_send_raw
```

was accepted by the mailbox and the same management sequence continued. Internal storage still did not appear in /proc/partitions.

Conclusion: A8 ANS is not using the newer ASC/RTKit mailbox layout. Linux can now communicate with the correct A8-era AKF mailbox, and the next implementation step is decoding/responding to the old RTBuddy management sequence, then starting ANSEndpoint1 and implementing ASP block commands.

2026-05-18 repeated AKF trace

Booted the AKF diagnostic payload again from DFU and captured a read-only trace without sending another raw command. Capture saved at:

```
artifacts/akf-trace-2026-05-18-1552.txt
```

Initial AKF mailbox state:

```
akf_set          @1000: 00001111
akf_clr          @1004: 00001111
akf_a2i_control  @1008: 0002cc01
akf_a2i_send0    @1010: 00000001
akf_a2i_send1    @1014: 00600000
akf_a2i_recv0    @1018: 000ff013
akf_a2i_recv1    @101c: 06000000
akf_i2a_control  @1020: 00029901
akf_i2a_recv0    @1038: 00000012
akf_i2a_recv1    @103c: 06000000
```

Three separate receive polls, including one-second waits between polls, repeated the same management sequence:

```
0600000000000012
0600000000000004
00b0000000000010
0070000000000001
```

The repeated result strengthens the interpretation that these are structured A8 RTBuddy/AppleA7IOPV1 management messages waiting for a correct response, not random register noise.

2026-05-18 decoder/guarded-reply payload

Added the next diagnostic step to drivers/soc/apple/t7000-ans-probe.c:

- akf_decode: read-only decoded polling of the AKF receive mailbox.
- akf_send_if_current: guarded send path that only writes a reply if the current receive message exactly matches the expected value supplied by the tester.

This gives us a safer way to test acknowledgements one at a time. Instead of blindly sending a raw message, the test command must include both:

```
expected-current-message reply-message
```

If the phone has moved to a different mailbox state, the driver skips the send.

Built payload:

```
artifacts/m1n1-linux-t7000-n56-akf-decode.bin
/tmp/m1n1-linux-t7000-n56-akf-decode.bin
```

The first decoder payload had a sysfs length accounting bug and printed a truncated line. Fixed payload:

```
artifacts/m1n1-linux-t7000-n56-akf-decode-fixed.bin
/tmp/m1n1-linux-t7000-n56-akf-decode-fixed.bin
```

Phone-side capture:

```
artifacts/akf-decode-fixed-2026-05-18-1930.txt
```

Decoded result:

```
msg=0600000000000012 hi=06000000 lo=00000012 top=06 rtkit_type=60 low16=0012 payload=000012
msg=0600000000000004 hi=06000000 lo=00000004 top=06 rtkit_type=60 low16=0004 payload=000004
msg=00b0000000000010 hi=00b00000 lo=00000010 top=00 rtkit_type=0b low16=0010 payload=000010
msg=0070000000000001 hi=00700000 lo=00000001 top=00 rtkit_type=07 low16=0001 payload=000001
```

The sequence repeated twice in a 16-sample decode window. This gives us a clean field split for the next guarded acknowledgement test.

2026-05-18 guarded AP-power echo test

Booted the fixed decoder payload again and tested exactly one guarded reply:

```
expected-current-message: 00b0000000000010
reply-message:            00b0000000000010
```

Command pattern:

```
echo 00b0000000000010 00b0000000000010 > \
  /sys/bus/platform/devices/208040000.ans/akf_send_if_current
```

The command was retried until the mailbox current value matched. The guard skipped the first six attempts with -EAGAIN, then sent once:

```
GUARDED_ECHO_SENT_7
```

Capture saved at:

```
artifacts/akf-guarded-echo-ap-power-2026-05-18-1936.txt
```

Observed result after the echo:

```
a2i_ctl changed from 0002cc01 to 0000cd01
sequence still repeated:
  0070000000000001
  0600000000000012
  0600000000000004
  00b0000000000010
```

No internal storage block device appeared in /proc/partitions.

Conclusion: echoing the 0x0b/0x10 AP-power-looking message is accepted by the AKF mailbox, but it is not the missing acknowledgement or next command needed to advance ANS to ANSEndpoint1/ASPStorage.

2026-05-18 guarded AP-on-after-IOP-ack test

Booted the fixed decoder payload again and tested one guarded state transition candidate:

```
expected-current-message: 0070000000000001
reply-message:            00b0000000000020
```

The send helper skipped until the current receive message exactly matched the expected 0x07/0x01 value, then sent once:

```
GUARDED_AP_ON_SENT_8
```

Capture saved at:

```
artifacts/akf-guarded-ap-on-after-iop-ack-2026-05-18-1950.txt
```

Observed mailbox state after the send:

```
akf_a2i_control @1008: 0000ce01
akf_a2i_send0   @1010: 00000020
akf_a2i_send1   @1014: 00b00000
akf_a2i_recv0   @1018: 00000020
akf_a2i_recv1   @101c: 00b00000
```

The controller accepted the message at the AKF mailbox level, but the receive stream returned to the same repeating management sequence:

```
0600000000000012
0600000000000004
00b0000000000010
0070000000000001
```

/proc/partitions still showed only RAM disks and the small MTD entries:

```
ram0..ram15
mtdblock0
mtdblock1
```

Conclusion: sending 00b0000000000020 after the 0070000000000001 message is also not sufficient to advance ANS into ANSEndpoint1/ASPStorage. The controlled send path is proven to work, but the missing piece is still the correct old-A8 RTBuddy/AKF management response sequence, likely around the 0600000000000012 / 0600000000000004 messages.

2026-05-18 guarded 0x06 management ack tests

Tested the simplest old-A8 interpretation of the two 0x06 management messages: treat them like IOP power/state messages and answer with matching 0x07 acknowledgements.

First test:

```
expected-current-message: 0600000000000012
reply-message:            0070000000000012
result:                   GUARDED_06_12_ACK_SENT_1
capture:                  artifacts/akf-guarded-ack-06-12-2026-05-18-1957.txt
```

Second test:

```
expected-current-message: 0600000000000004
reply-message:            0070000000000004
result:                   GUARDED_06_04_ACK_SENT_4
capture:                  artifacts/akf-guarded-ack-06-04-2026-05-18-1959.txt
```

Both messages were accepted by the AKF mailbox send path. After each send, the controller returned to the same repeating management sequence:

```
0600000000000012
0600000000000004
00b0000000000010
0070000000000001
```

/proc/partitions still did not show the internal NAND; only RAM disks and the two tiny MTD blocks were present.

Conclusion: the missing handshake is not a simple 0x06 -> 0x07 same-payload acknowledgement for either observed 0x06 message. The next step should be to derive the old A8 RTBuddy management sequence from iOS/m1n1 behavior instead of trying more one-off guessed acknowledgements.

2026-05-18 local iOS 12.5.8 driver study

Used the local iPhone 6 Plus iOS 12.5.8 restore files already on disk:

```
/Users/nicvotolato/Desktop/iPhone_5.5_12.5.8_16H88_Restore/kernelcache.release.iphone7
```

Decompressed local driver image:

```
analysis/ios1258-driver-study/kernelcache.dec/kernelcache.release.iphone7.decompressed
```

Confirmed storage-related modules in the image:

```
com.apple.driver.AppleNANDConfigAccess
```

```
com.apple.driver.AppleA7IOP
com.apple.driver.RTBuddy
com.apple.driver.ASPSupportNodes
com.apple.driver.AppleEffaceableStorage
com.apple.driver.AppleEffaceableBlockDevice
```

Important classes discovered:

```
AppleA7IOPV1
AppleA7IOPNub
RTBuddyManagementEndpoint
RTBuddyEndpoint
RTBuddyEndpointService
RTBuddyService
RTBuddyMailboxDecoder
RTBuddyRtkitDecoder
ASPStorage
ASPRequest
ASPFirmware
ASPLLBFirmware
ASPPanicLog
ASPEffaceable
ASPDiagnostic
AppleNANDConfigAccess
```

The iOS personality data confirms the stack shape:

```
AppleA7IOPV1 matches IONameMatch "iop,s5l8960x"
ASPStorage matches provider RTBuddyEndpointService
ASPStorage uses IONameMatch "ANSEndpoint1"
ASPBlockStorage, ASPFirmware, ASPEffaceable, ASPNVRAM, and others attach below ASPStorage
```

Interesting local source-reference strings:

```
RTBuddyManagementEndpoint.cpp:
  unsupported message received on management endpoint
  invalid management message ... status = ...
  Invalid pinger sequence number received
  unsupported power state

RTBuddy.cpp:
  Incompatible protocol version
  Failed to validate IOP: status=..., version=...
  No response received ... not started?

ASPStorage.cpp:
  ASP_PROTOCOL_VERSION mismatch
  ASPStorage::ExecuteCommand - sendMessage() failed
  Invalid unsolicited signal value
  Invalid message type received
  RTBuddy returned ... from setPowerState
```

Disassembly around RTBuddyManagementEndpoint shows protocol-format branching around a 0x40 selector and message-size choices such as 0x44/0x54 versus 0x110/0x120. This supports the current conclusion that the observed A8 management traffic is not handled by the simple modern compact RTKit sequence we were testing. The next implementation step should model the old RTBuddyManagementEndpoint layout enough to parse message headers, status, version, pinger, and power-state fields before attempting further replies.

2026-05-18 device run with expanded old-management decoder

Added an expanded read-only decoder to t7000-ans-probe.c and packaged:

```
artifacts/m1n1-linux-t7000-n56-akf-oldmgmt-decode.bin
/tmp/m1n1-linux-t7000-n56-akf-oldmgmt-decode.bin
```

The phone booted the diagnostic image and exposed the Hoolock Linux console. Capture saved at:

```
artifacts/akf-oldmgmt-decode-2026-05-18-2024.txt
```

Read-only state:

```
mmio_size=0x2000 pm_ret=0 runtime_status=active
```

Mailbox state:

```
akf_a2i_control  @1008: 0002cc01
akf_a2i_send0    @1010: 00000001
akf_a2i_send1    @1014: 00600000
akf_a2i_recv0    @1018: 000ff013
akf_a2i_recv1    @101c: 06000000
akf_i2a_control  @1020: 00029901
akf_i2a_recv0    @1038: 00000012
akf_i2a_recv1    @103c: 06000000
```

Expanded decoder confirmed the same repeating sequence, now labelled with candidate management fields:

```
0600000000000012 mgmt_type=60 major=0 sub=0 low16=0012 low8=12
0600000000000004 mgmt_type=60 major=0 sub=0 low16=0004 low8=04
00b0000000000010 mgmt_type=0b major=0 sub=0 low16=0010 low8=10
0070000000000001 mgmt_type=07 major=0 sub=0 low16=0001 low8=01
```

The device still did not expose internal NAND to Linux:

```
/proc/partitions:
ram0..ram15
mtdblock0
mtdblock1
```

Conclusion: the new run confirms the controller is active and the old-management traffic is stable, but no storage block device appears from decoding alone. The next code step is to implement a real old-RTBuddy management state machine: parse and track the 0x60, 0x0b, and 0x07 management events with state, status, version/pinger fields, then derive an ordered response sequence from the iOS RTBuddyManagementEndpoint behavior.

2026-05-18 old RTBuddy state-machine scaffold

Added a first Linux-side state-machine scaffold to drivers/soc/apple/t7000-ans-probe.c.

New sysfs view:

```
/sys/bus/platform/devices/208040000.ans/oldrtbuddy_state
```

The new view samples the AKF receive mailbox, classifies the observed A8 management events, and tracks:

```
phase
last_msg
```

```
seen state12/state04/ap_power10/iop_ack01
loop_count
repeat_state12
iop_state
ap_power_state
iop_ack_state
last mailbox control bits
```

The scaffold deliberately does not send replies yet. Its purpose is to make the driver treat the observed values as an ordered protocol state instead of isolated hex samples. That gives the next patch a place to add an ordered reply path and later an ANSEndpoint1 start path.

Built and packaged:

```
artifacts/m1n1-linux-t7000-n56-oldrtbuddy-state.bin
/tmp/m1n1-linux-t7000-n56-oldrtbuddy-state.bin
```

Kernel build result:

```
make LLVM=1 ARCH=arm64 Image.gz dtbs
completed successfully
```

2026-05-18 phone run with ordered reply build

Booted the ordered-reply build on the iPhone 6 Plus and confirmed the new sysfs path exists on the phone:

```
/sys/bus/platform/devices/208040000.ans/oldrtbuddy_plan
```

Capture saved at:

```
artifacts/ordered-reply-plan-2026-05-18-2238.txt
```

Phone-side result:

```
oldrtbuddy_ordered_reply=v1
stable=1 samples=64 unknown=0 phase=stable-management-loop last_msg=0070000000000001
loop_count=7 seen_mask=f
phases: state12 expects 0600000000000012, state04 expects 0600000000000004, ap_power10
expects 00b0000000000010, iop_ack01 expects 0070000000000001
guard=requires stable four-message management loop, matching current phase, exact expected
message, and non-full AP-to-IOP mailbox
```

The state-machine view still sees the same stable loop:

```
oldrtbuddy_state=v1
samples=64 unknown=0 phase=stable-management-loop last_msg=0070000000000001
seen: state12=1 state04=1 ap_power10=1 iop_ack01=1 mask=f
loop_count=7 repeat_state12=32 iop_state=0004 ap_power_state=0010 iop_ack_state=0001
```

Storage result is still unchanged:

```
/proc/partitions:
ram0..ram15
mtdblock0
mtdblock1
```

Conclusion: the ordered state-machine reply path is now live on the phone. No reply was sent in this run. The next experiment should choose a specific management acknowledgement sequence and send it through oldrtbuddy_ordered_reply

so the driver refuses to write unless the controller is at the matching phase.

2026-05-19 first ordered-reply test and guard improvement

Booted the ordered-reply image again and tested a canonical/type-6 byte-order candidate through the guarded sysfs path:

```
echo 'ap_power10 0060000000000010' >
/sys/bus/platform/devices/208040000.ans/oldrtbuddy_ordered_reply
```

Capture saved at:

```
artifacts/oldrtbuddy-ordered-reply-2026-05-19-181808.txt
```

The write returned:

```
sh: write error: Resource temporarily unavailable
SEND_STATUS 1
```

This means the guard refused to send. The pre-send state was still good:

```
oldrtbuddy_ordered_reply=v1
stable=1 samples=64 unknown=0 phase=stable-management-loop
last_msg=0070000000000001 loop_count=7 seen_mask=f
```

The likely reason is that oldrtbuddy_ordered_reply first sampled the stable loop and then checked only one instant of the rotating mailbox. By the time the write path checked the current value, the mailbox was no longer exactly at ap_power10, so it correctly returned -EAGAIN instead of sending.

Updated drivers/soc/apple/t7000-ans-probe.c so oldrtbuddy_ordered_reply now:

1. Confirms the full stable four-message loop.
2. Waits briefly for the requested phase to appear.
3. Sends only if the live message exactly matches the requested phase.
4. Logs wait_samples when it sends.

Built and packaged the updated image:

```
artifacts/m1n1-linux-t7000-n56-ordered-reply-wait.bin
/tmp/m1n1-linux-t7000-n56-ordered-reply-wait.bin
```

Kernel build result:

```
make LLVM=1 ARCH=arm64 Image.gz dtbs
completed successfully
```

The phone then reported normal iOS mode, so the updated image still needs a fresh DFU run before the canonical/type-6 candidate can be tested for real.

2026-05-19 ordered phase-wait send test

Booted the updated phase-wait image:

```
/tmp/m1n1-linux-t7000-n56-ordered-reply-wait.bin
```

Ran the same canonical/type-6 byte-order candidate again:

```
echo 'ap_power10 0060000000000010' >
/sys/bus/platform/devices/208040000.ans/oldrtbuddy_ordered_reply
```

Capture saved at:

```
artifacts/oldrtbuddy-ordered-reply-2026-05-19-182724.txt
```

This time the guarded write succeeded:

```
SEND_ORDERED_REPLY ap_power10 0060000000000010
SEND_STATUS 0
```

Before the send, the controller was in the known stable loop:

```
oldrtbuddy_ordered_reply=v1
stable=1 samples=64 unknown=0 phase=stable-management-loop
last_msg=0070000000000001 loop_count=7 seen_mask=f
```

After the send, the controller was still in the same management loop:

```
oldrtbuddy_ordered_reply=v1
stable=1 samples=64 unknown=0 phase=ap-power-0x10
last_msg=00b0000000000010 loop_count=8 seen_mask=f
```

Storage did not appear:

```
/proc/partitions:
ram0..ram15
mtdblock0
mtdblock1
```

Post-check capture:

```
artifacts/oldrtbuddy-ordered-reply-postcheck-2026-05-19-182836.txt
```

akf_decode after the send still showed the same four-message cycle:

```
0070000000000001
0600000000000012
0600000000000004
00b0000000000010
```

Conclusion: the ordered phase-wait send path works, and the canonical/type-6 reply candidate was accepted by the mailbox, but it did not advance ANS to ANSEndpoint1 or expose internal NAND. The blocker is now more specific: Linux needs the real old RTBuddy management transaction format, likely more than one 64-bit acknowledgement value.

2026-05-19 ordered reply matrix

Booted the phase-wait ordered-reply image again and ran a small guarded matrix of plausible single-value responses. Capture saved at:

```
artifacts/oldrtbuddy-reply-matrix-2026-05-19-184300.txt
```

Every write went through oldrtbuddy_ordered_reply, so each send was gated by:

1. stable four-message loop observed first,
2. requested phase seen live,
3. exact current message matched for that phase,
4. AP-to-IOP mailbox not full.

Tested:

```
ap_power10 0060000000000010  canonical/type6 ap-power candidate
ap_power10 00b0000000000010  old-layout ap-power echo
ap_power10 00b0000000000020  old-layout ap-power next-state candidate
iop_ack01  0070000000000001  old-layout iop-ack echo
iop_ack01  0060000000000001  canonical/type6 iop-ack candidate
state12    0060000000000012  canonical/type6 state12 candidate
state04    0060000000000004  canonical/type6 state04 candidate
```

All seven writes returned success:

```
MATRIX_STATUS_1 0
MATRIX_STATUS_2 0
MATRIX_STATUS_3 0
MATRIX_STATUS_4 0
MATRIX_STATUS_5 0
MATRIX_STATUS_6 0
MATRIX_STATUS_7 0
```

None of them advanced ANS out of the same management loop. After each send, oldrtbuddy_state / oldrtbuddy_plan still reported:

```
stable=1
unknown=0
seen_mask=f
```

akf_decode still showed the same repeating messages:

```
0600000000000012
0600000000000004
00b0000000000010
0070000000000001
```

Storage still did not appear:

```
/proc/partitions:
ram0..ram15
mtdblock0
mtdblock1
```

Conclusion: the mailbox send path is working and can safely target specific old RTBuddy phases, but the obvious single-word responses are insufficient. The remaining blocker is almost certainly the higher-level old RTBuddy transaction structure: message headers, payload buffers, sequence/status fields, or endpoint-map/start exchanges rather than one standalone 64-bit value.

2026-05-19 guarded transaction-plan phone run

Added a guarded multi-step transaction test path to drivers/soc/apple/t7000-ans-probe.c and booted it on the iPhone 6 Plus.

New sysfs files:

```
/sys/bus/platform/devices/208040000.ans/oldrtbuddy_transaction_plan
/sys/bus/platform/devices/208040000.ans/oldrtbuddy_transaction
```

Built and booted:

```
artifacts/m1n1-linux-t7000-n56-oldrtb-transaction.bin
/tmp/m1n1-linux-t7000-n56-oldrtb-transaction.bin
```

Capture saved at:

```
artifacts/oldrtbuddy-transaction-plans-2026-05-19-191213.txt
```

Initial state after boot was unchanged and stable:

```
stable=1 samples=64 unknown=0 phase=stable-management-loop
last_msg=0070000000000001 loop_count=7 seen_mask=f
```

The transaction plans available in this build were:

```
canonical4
oldecho4
mixed-power-first
mixed-state-first
```

Results:

```
TRANSACTION_STATUS_1 canonical4        0
TRANSACTION_STATUS_2 oldecho4          0
TRANSACTION_STATUS_3 mixed-power-first 1, Resource busy
TRANSACTION_STATUS_4 mixed-state-first 1, Resource busy
```

canonical4 and oldecho4 successfully wrote their guarded multi-step sequences, but neither advanced the controller out of the old RTBuddy management loop. Internal NAND still did not appear:

```
/proc/partitions:
ram0..ram15
mtdblock0
mtdblock1
```

After the successful plans, later sends hit Resource busy because the AP-to-IOP AKF send mailbox became full:

```
last_controls: a2i=0001c401 i2a=00091901 i2a_empty=0 i2a_full=1
```

Conclusion: the guarded multi-step path also does not reach ANSEndpoint1. This is useful negative evidence. The remaining work should move away from more 64-bit mailbox-word combinations and toward the real old RTBuddy transaction packet/buffer path: header, length, sequence/status fields, endpoint map, endpoint start, then ASPStorage/ASPBlockStorage.

2026-05-19 old RTBuddy packet layout model

Implemented the first code-side model for the higher-level old RTBuddy management packet format in drivers/soc/apple/t7000-ans-probe.c.

This is based on the local iOS 12.5.8 RTBuddyManagementEndpoint disassembly around:

```
0xfffffff0080baa98
```

The modeled packet layout is:

```
packet + 0x00: 8-byte prefix copied by iOS from kernel rodata
packet + 0x08: word0
packet + 0x0c: word1
packet + 0x10: payload
```

The fixed prefix was recovered from the decompressed iOS 12.5.8 iPhone7 kernelcache. The disassembly loads it from:

```
0xffffff0070cb080
```

Address-to-file offset:

```
0xc7080
```

Bytes:

```
04 00 00 00 54 00 00 00
```

Two variants are now represented in the driver:

```
old-small:
  format      0x00
  word0       0x00000006
  word1       0x00000011
  payload_len 0x44
  total_len   0x54
old-0x40:
  format      0x40
  word0       0x00000007
  word1       0x00000044
  payload_len 0x110
  total_len   0x120
```

New sysfs files:

```
/sys/bus/platform/devices/208040000.ans/oldrtbuddy_packet_model
/sys/bus/platform/devices/208040000.ans/oldrtbuddy_packet_stage
/sys/bus/platform/devices/208040000.ans/oldrtbuddy_packet_dump
```

Usage:

```
cat /sys/bus/platform/devices/208040000.ans/oldrtbuddy_packet_model
echo old-small > /sys/bus/platform/devices/208040000.ans/oldrtbuddy_packet_stage
cat /sys/bus/platform/devices/208040000.ans/oldrtbuddy_packet_dump
echo old-0x40 > /sys/bus/platform/devices/208040000.ans/oldrtbuddy_packet_stage
cat /sys/bus/platform/devices/208040000.ans/oldrtbuddy_packet_dump
```

Important: this does not send the packet to hardware yet. It stages and dumps the inferred packet bytes inside the driver. That is deliberate. The next missing piece is identifying the real shared-buffer address/path used by the old A8 RTBuddy endpoint before packet writes are enabled.

Built successfully:

```
make LLVM=1 ARCH=arm64 Image.gz dtbs
```

Packaged:

```
artifacts/m1n1-linux-t7000-n56-oldrtb-packet-model.bin
/tmp/m1n1-linux-t7000-n56-oldrtb-packet-model.bin
```

Device validation:

```
artifacts/oldrtbuddy-packet-model-2026-05-19-205548.txt
```

The packet-model image booted on the iPhone 6 Plus and exposed the new packet sysfs files:

```
/sys/bus/platform/devices/208040000.ans/oldrtbuddy_packet_model
/sys/bus/platform/devices/208040000.ans/oldrtbuddy_packet_stage
/sys/bus/platform/devices/208040000.ans/oldrtbuddy_packet_dump
```

Both modeled packet variants staged successfully:

```
STAGE_OLD_SMALL_STATUS 0
oldrtbuddy_packet_dump=v1 seq=1 format=0x00 len=0x54
prefix_status=unknown-rodata-zero-filled word0_le=06000000 word1_le=11000000
```

```
STAGE_OLD_40_STATUS 0
oldrtbuddy_packet_dump=v1 seq=2 format=0x40 len=0x120
prefix_status=unknown-rodata-zero-filled word0_le=07000000 word1_le=44000000
```

The controller state after staging was still the stable old RTBuddy management loop:

```
oldrtbuddy_state=v1
samples=64 unknown=0 phase=stable-management-loop last_msg=0070000000000001
seen: state12=1 state04=1 ap_power10=1 iop_ack01=1 mask=f
loop_count=7
```

Storage result was unchanged:

```
/proc/partitions:
ram0..ram15
mtdblock0
mtdblock1
```

Conclusion: the higher-level packet-builder path now exists and has been tested on the phone. This still does not expose NAND because the staged buffer is not yet a full old A8 RTBuddy transaction. The fixed iOS prefix is now known, but the payload is still a placeholder:

```
prefix_le=0400000054000000
payload=diagnostic pattern
```

The next code step is to derive the real payload/shared-buffer path, then replace the diagnostic pattern with a specific endpoint-map/start transaction before enabling any guarded hardware send.

Implementation update:

```
drivers/soc/apple/t7000-ans-probe.c now stages prefix_le=0400000054000000
```

2026-05-19 DMA-backed packet staging

The next implementation step is now represented in code: the old RTBuddy packet model no longer only lives in a normal kernel byte array. When oldrtbuddy_packet_stage is used, the driver allocates a coherent DMA packet buffer and copies the staged packet into it.

New behavior:

```
echo old-small > /sys/bus/platform/devices/208040000.ans/oldrtbuddy_packet_stage
cat /sys/bus/platform/devices/208040000.ans/oldrtbuddy_packet_dump
```

Expected new diagnostic fields:

```
dma_packet allocated=1 dma=<device-visible address> size=0x120 copied_len=0x54
prefix_le=0400000054000000
```

For old-0x40, the copied length should be:

```
copied_len=0x120
```

Important: this still does not send the DMA address/index to the storage controller. It proves the Linux side can create a device-visible shared packet buffer with the modeled old RTBuddy header. The missing step is still the exact old RTBuddy mailbox command that tells ANS which shared buffer to consume, plus the real payload contents instead of the current diagnostic pattern.

Build note: direct macOS rebuilding needs a no-space source path and Linux-style host headers for kernel helper tools. The local macOS attempt reached host tool compilation but then stopped in scripts/mod/modpost.c because the macOS/libelf header path lacks several Linux ELF relocation constants. This was bypassed by building inside an Ubuntu Docker container mounted at /work.

Docker build result:

```
make LLVM=1 ARCH=arm64 Image.gz dtbs
```

Completed successfully with the DMA-backed packet staging code.

Packaged:

```
artifacts/m1n1-linux-t7000-n56-oldrtb-dma-packet.bin
/tmp/m1n1-linux-t7000-n56-oldrtb-dma-packet.bin
```

Current phone state check after packaging showed the phone was still running the previous packet-model build:

```
layout: +0x00 prefix[8] unknown-rodata
guard=this only stages bytes locally; no hardware send occurs until shared buffer addressing
 is identified
```

So the DMA-backed packet staging image still needs to be booted on the phone for the next validation.

2026-05-20 physical packet-buffer fallback phone run

Booted:

```
/tmp/m1n1-linux-t7000-n56-oldrtb-phys-packet.bin
```

Capture saved at:

```
artifacts/oldrtbuddy-phys-packet-2026-05-20-000806.txt
```

The phone booted the fallback build:

```
Linux (none) 7.1.0-rc3 #16 SMP PREEMPT Wed May 20 04:03:51 UTC 2026 aarch64 Linux
```

The old RTBuddy packet model now reports the recovered iOS prefix:

```
layout: +0x00 prefix[8]=0400000054000000, +0x08 word0, +0x0c word1, +0x10 payload
```

The normal coherent DMA allocation failed in the earlier #15 build, so this build falls back to a zeroed physical page. That fallback worked on the phone.

old-small staged successfully:

```
STAGE_OLD_SMALL_PHYS_STATUS 0
oldrtbuddy_packet_dump=v1 seq=1 format=0x00 len=0x54
dma_packet allocated=1 mode=phys-fallback addr=0x0000000806476000 size=0x1000
copied_len=0x54
prefix_le=0400000054000000 word0_le=06000000 word1_le=11000000 payload_status=diagnostic-
```

```
pattern
```

old-0x40 staged successfully into the same physical packet buffer:

```
STAGE_OLD_40_PHYS_STATUS 0
oldrtbuddy_packet_dump=v1 seq=2 format=0x40 len=0x120
dma_packet allocated=1 mode=phys-fallback addr=0x0000000806476000 size=0x1000
copied_len=0x120
prefix_le=0400000054000000 word0_le=07000000 word1_le=44000000 payload_status=diagnostic-
pattern
```

This is the first phone run where Linux created a device-addressable old RTBuddy packet buffer containing the recovered iOS header.

Storage result is still unchanged:

```
/proc/partitions:
ram0..ram15
mtdblock0
mtdblock1
```

Controller state after staging remained the stable old RTBuddy management loop:

```
oldrtbuddy_state=v1
samples=64 unknown=0 phase=stable-management-loop last_msg=0070000000000001
seen: state12=1 state04=1 ap_power10=1 iop_ack01=1 mask=f
loop_count=7
```

Conclusion: the project has moved from "we can model a packet" to "Linux can place the modeled packet at a real physical address on the phone." The next blocker is identifying the exact old RTBuddy mailbox transaction that tells ANS to consume that buffer, and replacing the diagnostic payload with the real endpoint-map/start payload.

2026-05-18 device run with old RTBuddy state-machine scaffold

Booted:

```
/tmp/m1n1-linux-t7000-n56-oldrtbuddy-state.bin
```

Capture saved at:

```
artifacts/oldrtbuddy-state-2026-05-18-2220.txt
```

The new state-machine sysfs view worked:

```
oldrtbuddy_state=v1
samples=64 unknown=0 phase=stable-management-loop last_msg=0070000000000001
seen: state12=1 state04=1 ap_power10=1 iop_ack01=1 mask=f
loop_count=7 repeat_state12=32 iop_state=0004 ap_power_state=0010 iop_ack_state=0001
last_controls: a2i=0002cc01 i2a=00088901 i2a_empty=0 i2a_full=0
```

Interpretation from the driver:

```
controller is alive but still cycling in old RTBuddy management; endpoint map/start not
reached
```

Storage result is unchanged:

```
/proc/partitions:
ram0..ram15
mtdblock0
```

```
mtdblock1
```

Conclusion: Linux now has a reproducible state-machine view of the old A8 management loop. The next patch should add an ordered reply path attached to this state machine. That path should be conservative: only send when the state machine has observed the full stable loop and the current message matches the expected phase.

2026-05-18 ordered old RTBuddy reply path

Added the first ordered reply mechanism to drivers/soc/apple/t7000-ans-probe.c.

New sysfs files:

```
/sys/bus/platform/devices/208040000.ans/oldrtbuddy_plan
/sys/bus/platform/devices/208040000.ans/oldrtbuddy_ordered_reply
```

oldrtbuddy_plan re-samples the AKF mailbox and reports whether the known four-message management loop is stable. It also prints the accepted phase names:

```
state12     expects 0600000000000012
state04     expects 0600000000000004
ap_power10  expects 00b0000000000010
iop_ack01   expects 0070000000000001
```

oldrtbuddy_ordered_reply is intentionally stricter than the older akf_send_if_current helper. It will only write a reply if all of these are true:

1. The driver observes the full four-message management loop.
2. No unknown AKF management messages appear in the sample window.
3. The requested phase matches the live current message.
4. The live current message exactly equals the expected value for that phase.
5. The AP-to-IOP AKF send mailbox is not full.

Example form:

```
echo 'ap_power10 00b0000000000020' >
/sys/bus/platform/devices/208040000.ans/oldrtbuddy_ordered_reply
```

This does not solve storage yet and does not invent a new RTBuddy response sequence. It gives the next phone-side run a controlled way to test ordered management acknowledgements without sending against the wrong phase of the controller loop.

Built and packaged:

```
artifacts/m1n1-linux-t7000-n56-ordered-reply.bin
/tmp/m1n1-linux-t7000-n56-ordered-reply.bin
```

Kernel build result:

```
make LLVM=1 ARCH=arm64 Image.gz dtbs
completed successfully
```

2026-05-20 guarded packet-address candidate run

Added a guarded packet-address test path to

drivers/soc/apple/t7000-ans-probe.c.

New sysfs files:

```
/sys/bus/platform/devices/208040000.ans/oldrtbuddy_packet_addr_plan
/sys/bus/platform/devices/208040000.ans/oldrtbuddy_packet_addr_send
```

The new sender stages the recovered old RTBuddy packet first, confirms the four-message management loop is stable, checks that the AKF AP-to-IOP mailbox is not full, then sends exactly one named address candidate.

Tested on phone with kernel:

```
Linux (none) 7.1.0-rc3 #17 SMP PREEMPT Wed May 20 19:28:34 UTC 2026 aarch64 Linux
```

The staged packet buffer used the physical-address fallback:

```
addr=0x000000080543a000
old-small len=0x54
old-0x40 len=0x120
```

Candidates tested in one boot:

```
old-small addr64          -> msg=000000080543a000
old-0x40 addr64           -> msg=000000080543a000
old-0x40 addr-shift4      -> msg=0000000080543a00
old-0x40 addr-shift12     -> msg=000000000080543a
old-0x40 addr-len         -> msg=000001200543a000
old-0x40 len-addr         -> msg=0543a00000000120
```

All writes returned success from the Linux-side guarded path, but the controller remained in the same stable management loop:

```
oldrtbuddy_state=v1
samples=64 unknown=0 phase=stable-management-loop last_msg=0070000000000001
seen: state12=1 state04=1 ap_power10=1 iop_ack01=1 mask=f
```

Storage result remained unchanged:

```
ram0..ram15
mtdblock0 16 KB
mtdblock1 128 KB
```

Conclusion: the missing transaction is not a plain physical address, shifted address, or simple address/length pair sent through the 64-bit AKF mailbox. The phone keeps answering, so the controller is alive, but it is not consuming the staged packet from these address candidate messages.

The next real engineering step is to reconstruct the old RTBuddy shared-buffer transaction structure around the iOS callback path: packet header, callback arguments, sequence/status fields, endpoint-map buffer, and ANSEndpoint1 start request. The evidence now points away from raw mailbox values and toward the higher-level buffer transaction format.

Capture:

```
artifacts/oldrtbuddy-packet-addr-candidates-2026-05-20-153508.txt
```